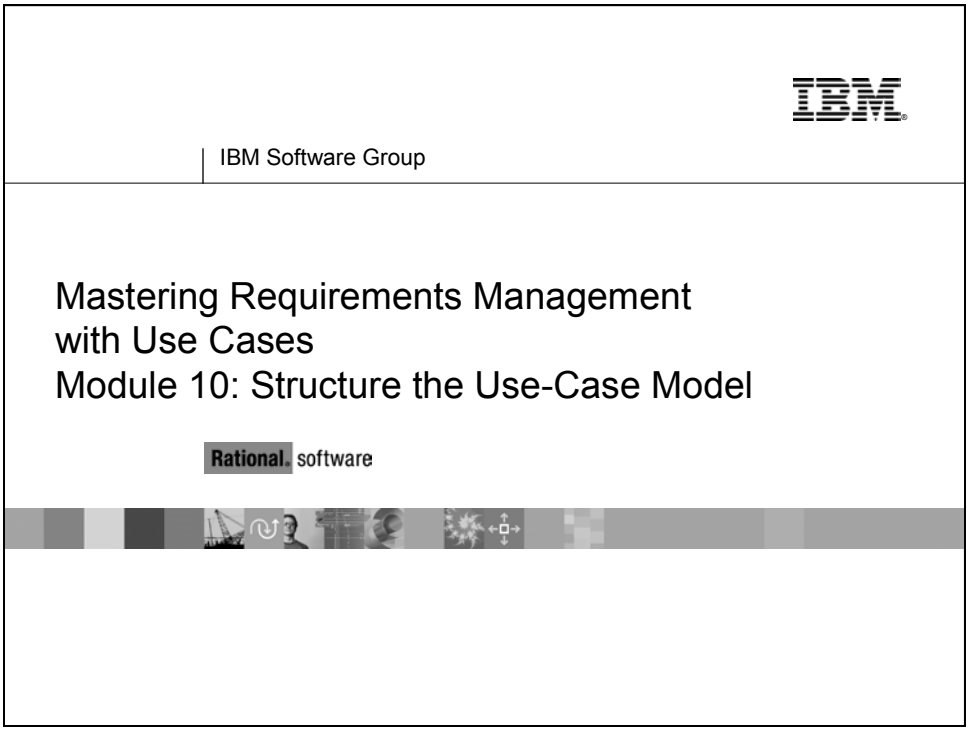
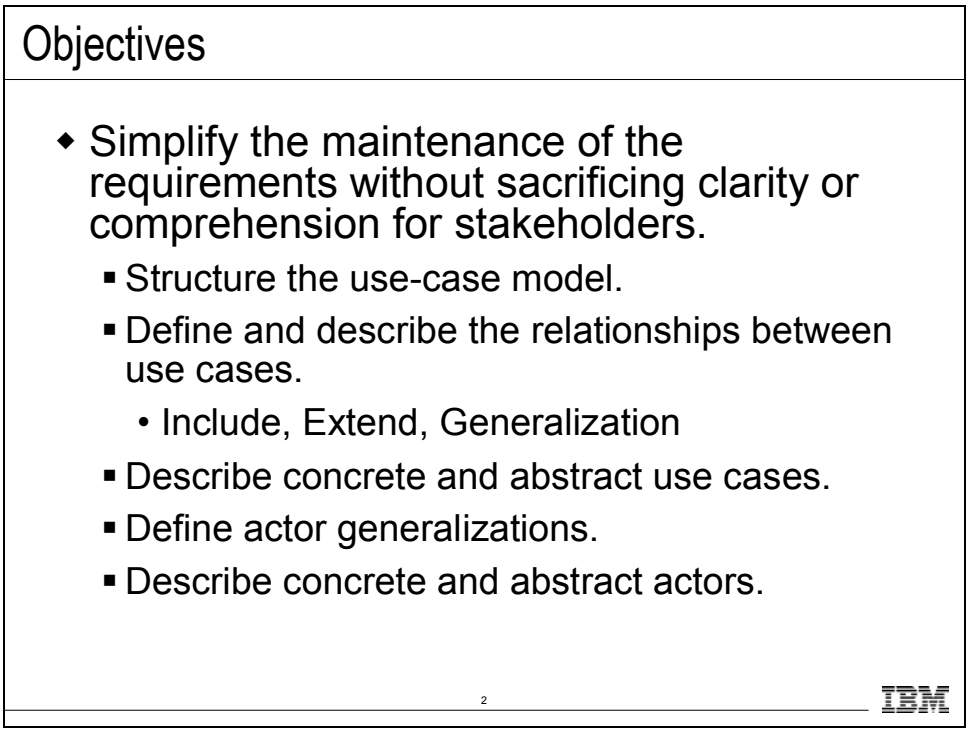
▸ ▸ ▸ Module 10
Structure the Use-Case Model



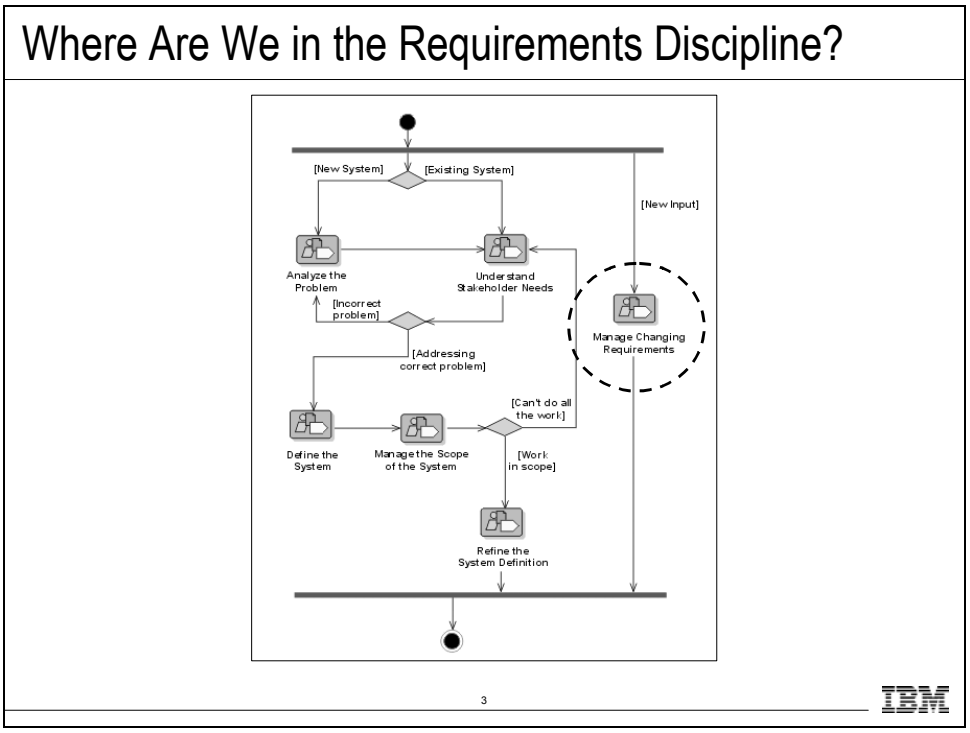
Topics

Objectives



An iterative process allows you to refine the model in smaller increments at each iteration and helps you maintain control of the changes throughout the product lifecycle.

Where Are We in the Requirements Discipline?

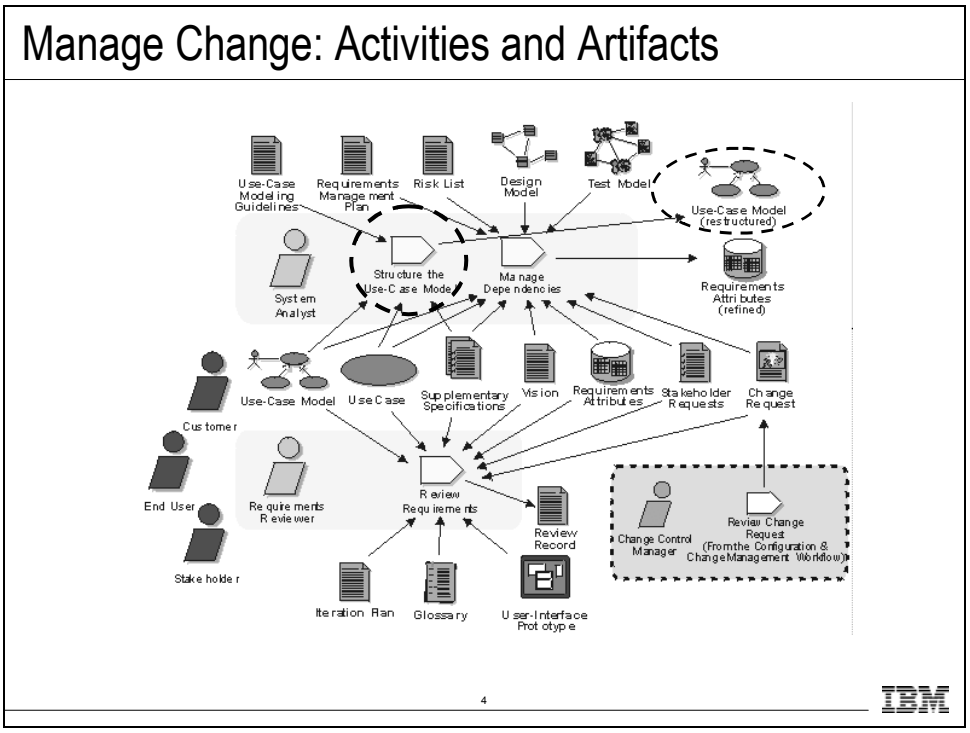


Where does Managing Changing Requirements workflow detail fit in the development process?

The workflow detail called "Manage Changing Requirements" in the Rational Unified Process represents activities we might undertake to help you manage change throughout the lifecycle of the system's development.

Where is this done in your own software development process?

Manage Change: Activities and Artifacts



The purpose of this workflow detail is to:

- Evaluate formally submitted change requests and determine their impact on the existing requirement set.
- Structure the use-case model.
- Set up appropriate requirements attributes and traceability.
- Formally verify that the results of the Requirements discipline conform to the customer's view of the system.

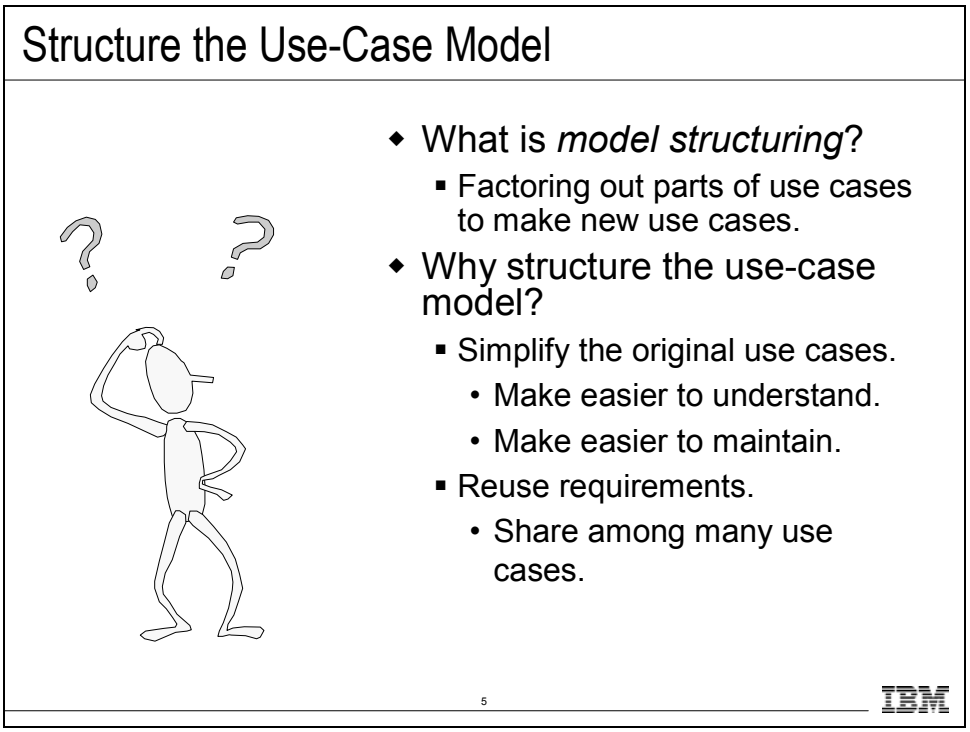
Changes to requirements naturally influence the models produced in the Analysis & Design discipline, as well as the test model created as part of the Test discipline.

Traceability relationships between the requirements identified in the manage dependency activity of this discipline and others is the key to understanding these effects.

Another important concept is the tracking of requirement history. By capturing the nature and rationale of requirement changes, reviewers (in this case the role is played by anyone on the software project team whose work is affected by the change) receive the information needed to respond to the change properly.

For this module, you focus on structuring the use-case model.

Structure the Use-Case Model



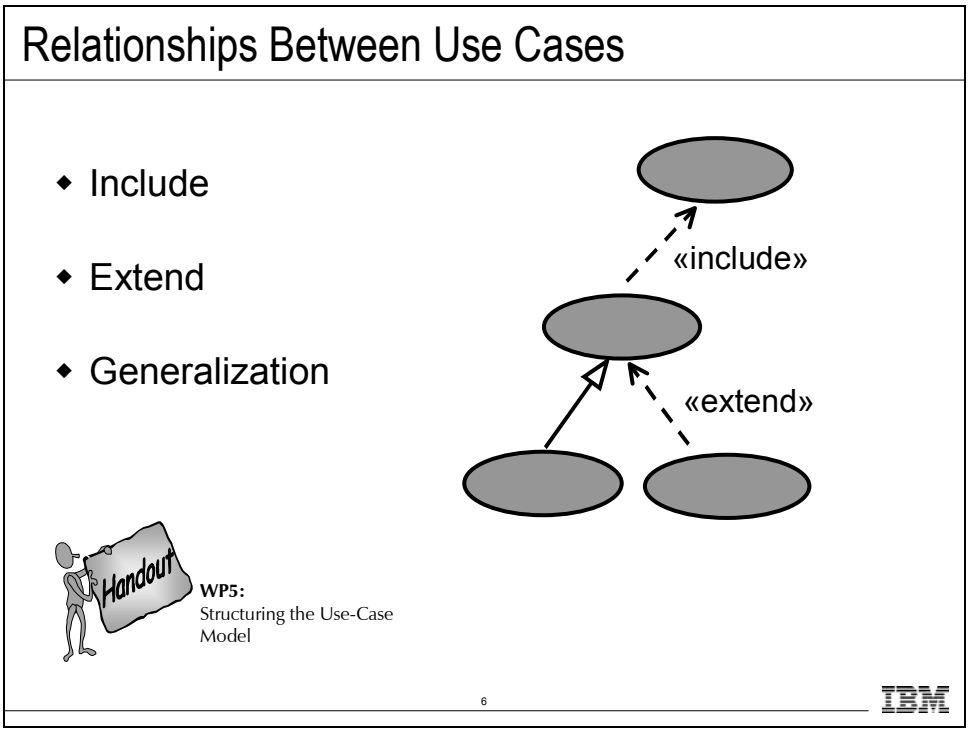
The single most important reason for structuring a use-case model is to achieve effective reuse of requirements without sacrificing clarity or comprehension. Thereby simplifying their maintenance.

Structuring the use-case model can help you handle changing requirements, such as added functionality or isolating changes to a particular requirement.

It is important not to start structuring the use-case model until you have a set of requirements the user understands and agrees to.

Make sure you do not start these too early, or you may confuse the customer (or yourself).

Relationships Between Use Cases



The purpose of structuring the use-case model is to extract behavior from one use case that can better be represented in a separate use case. Examples of behavior to factor out are:

- Common behavior
- Optional behavior
- Exceptional behavior
- Behavior that is to be developed in later iterations.

To structure the use cases, there are three different kinds of relationships:

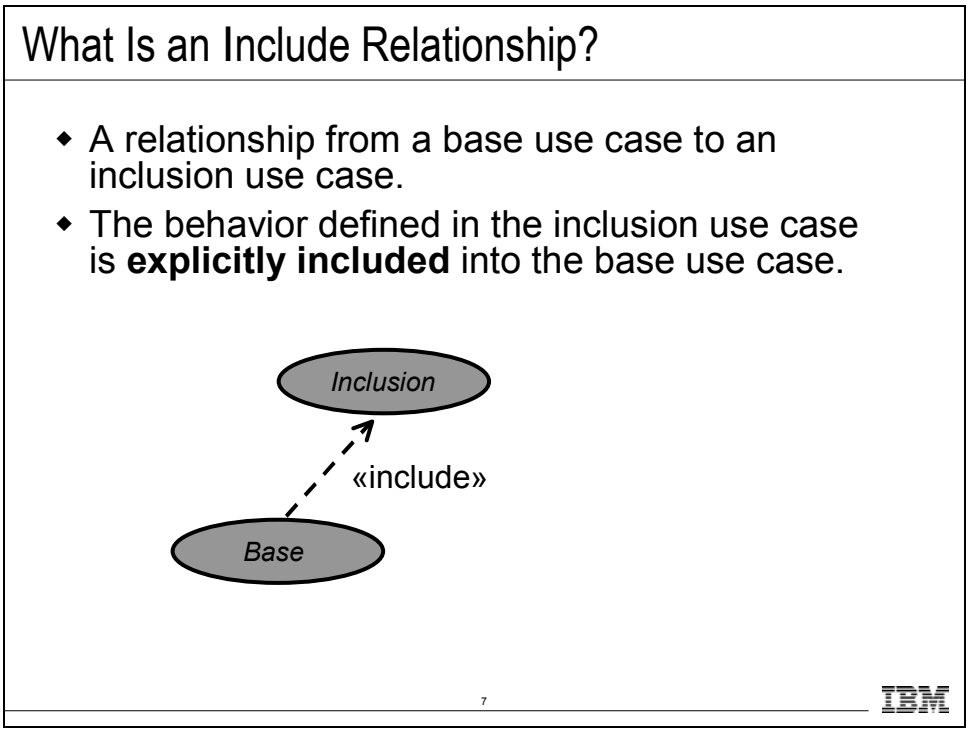
- Include relationship
- Extend relationship
- Generalization relationship

The use case that represents the modification is called the addition use case. The original use case that is modified is called the base use case.

We give definitions and examples of each type of relationship on the following slides.

For reference and for further information, see the white paper *Structuring the Use-Case Model*.

What Is an Include Relationship?

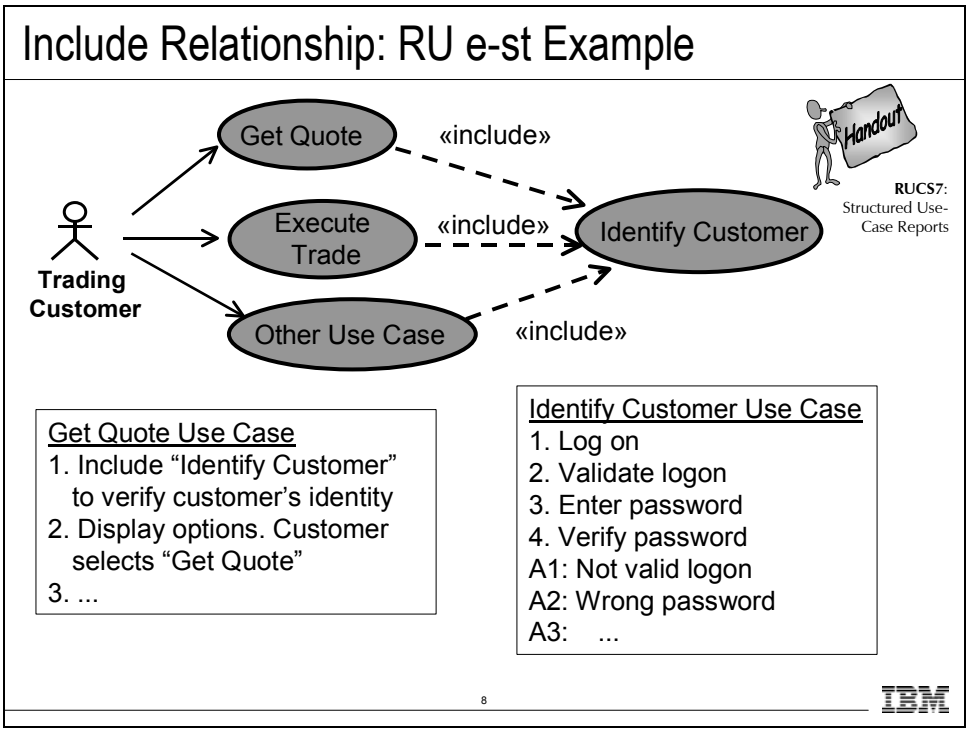


The include relationship connects a base use case to an inclusion use case. The inclusion use case describes a behavior segment that is inserted into a use-case instance that is executing the base use case.

The base use case explicitly includes the inclusion use case. The base use case can depend on the result of performing the inclusion, but neither the base nor the inclusion may access each other's attributes.

The inclusion in this sense is encapsulated and represents behavior that can be reused in different base use cases.

Include Relationship: RU e-st Example



In this example, the Identify Customer use case contains all of the behavior that involves logging on to the system and entering a password. This behavior can then be extracted from all of the use cases that require customer identification, such as:

Execute Trade
Transfer Between Accounts
Transfer With Banks

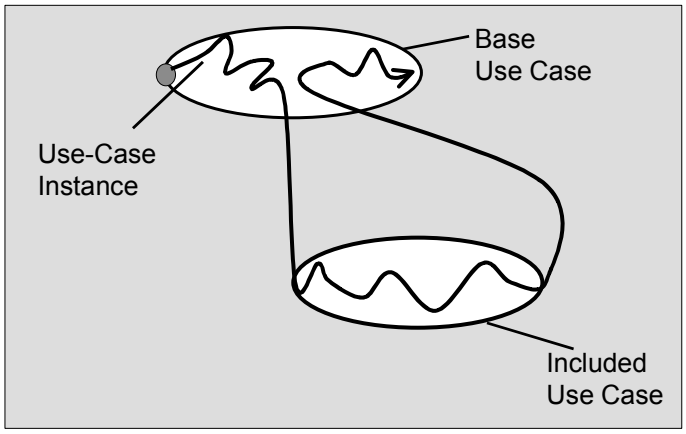
Each of these use cases now simply «include» the new use case, Identify Customer.

Note: Overuse of «extend» and «include» leads to a model that is functionally decomposed and therefore harder to test and implement.

Execution of an Include Relationship

Execution of an Include Relationship

- Fully executed when the inclusion point is reached.



9

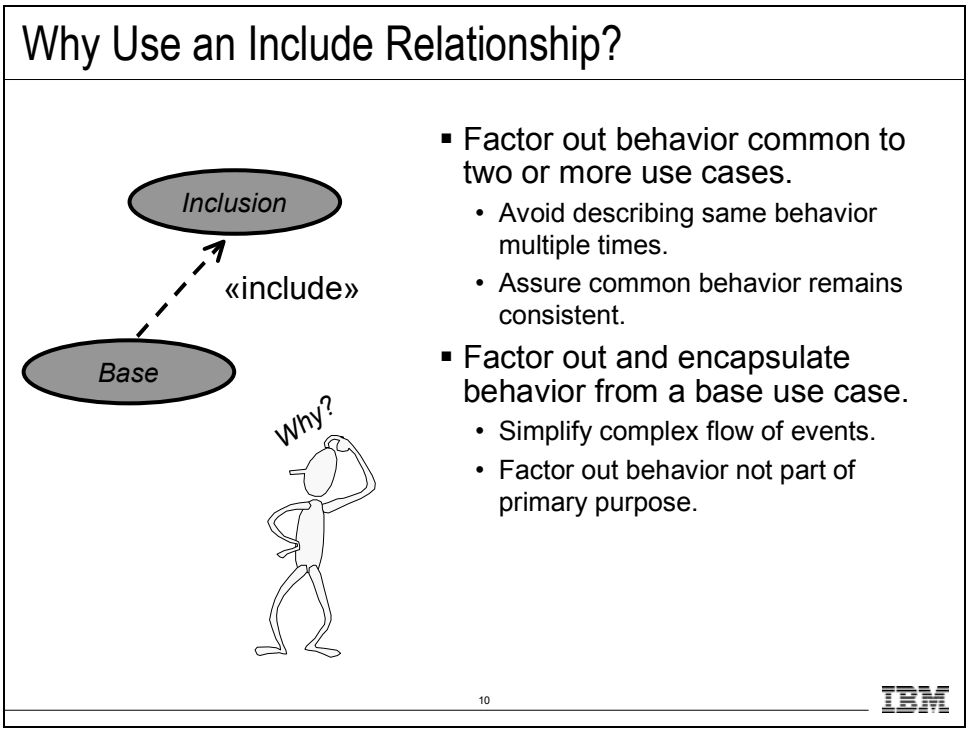
IBM

The behavior of the inclusion is inserted in one location in the base use case. When a use-case instance following the description of a base use case reaches a location in the base use case from which include relationship is defined, it instead follows the description of the inclusion use case. Once the inclusion is performed, the use-case instance resumes where it left off in the base use case.

The include-relationship is not conditional. If a use-case instance reaches the location of the "include" statement in the base use case, the included use case is always executed. But if a use-case instance never reaches the location in the base use case where the include relationship is defined, the included use case is not executed.

If you want to express a condition that only sometimes includes a use case, you need to place the include statement in an alternative flow of the base use case.

Why Use an Include Relationship?



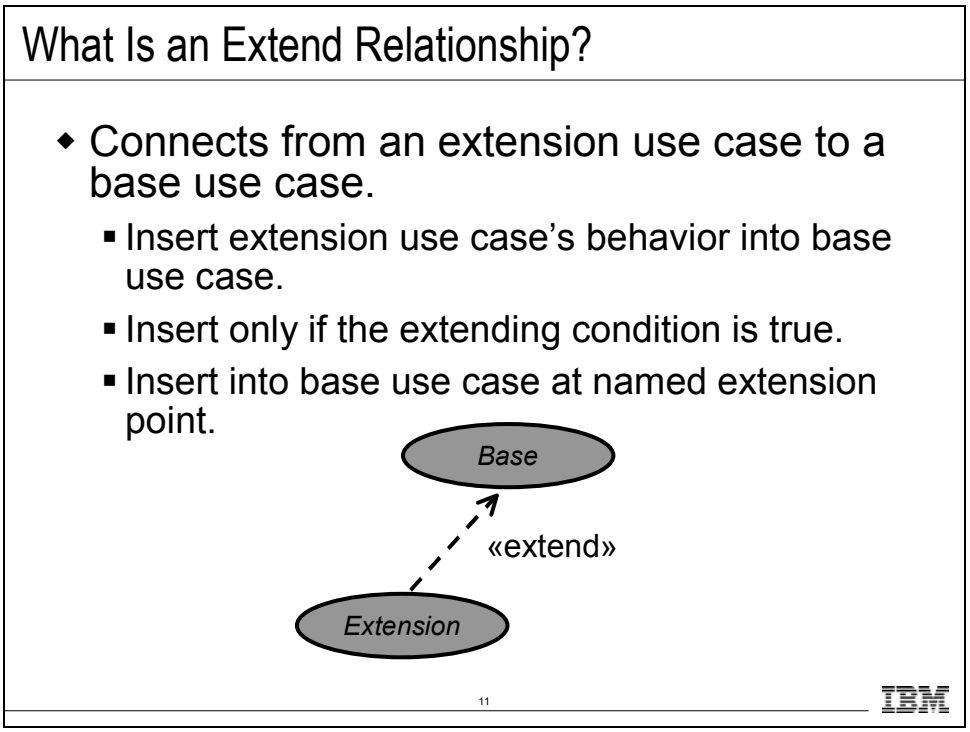
If several use cases share the same behavior or a particular subflow is overly complex, then you can factor out the shared or complex behavior into an inclusion use case.

If a part of a base use case represents a function of which the use case only depends on the result and not the method used to produce the result, you can factor that part out to an addition use case. The addition is explicitly inserted in the base use case, using the include relationship.

Some organizations use a rule of thumb that every flow of events should be no more than a certain length, typically one page. If a flow gets to be longer than one page, you can often shorten it by doing the following:

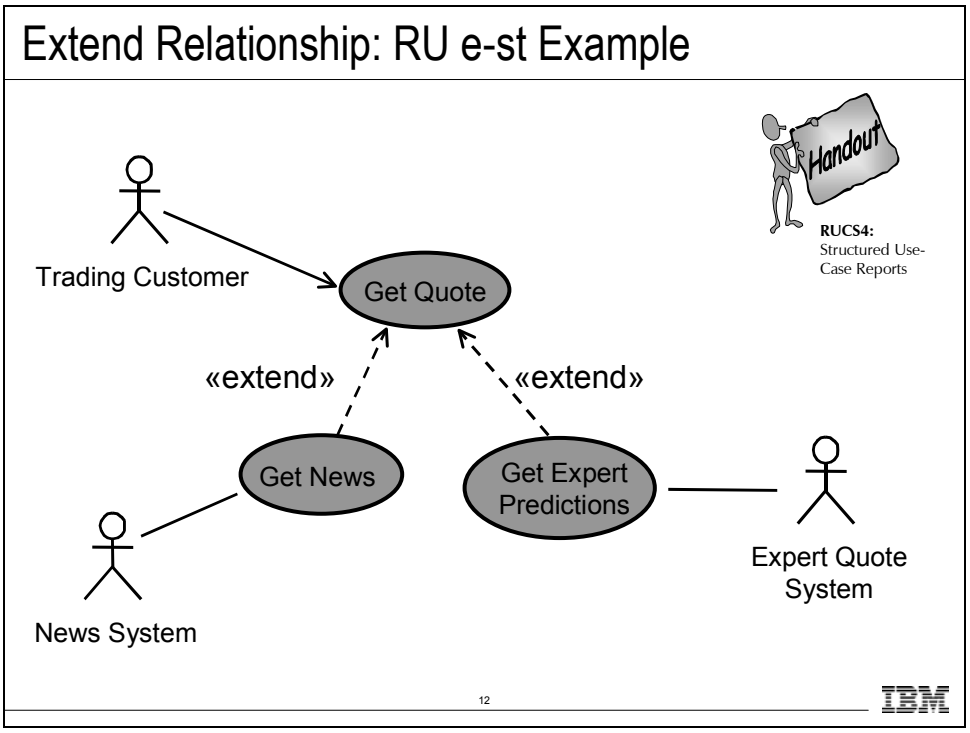
- Identify a subflow that has its own goal and coherent set of steps.
- Remove the subflow from the original flow.
- Encapsulate the subflow in a new use case.
- «include» the new use case in the original flow.

What Is an Extend Relationship?



The extend relationship connects an extension use case to a base use case. You define where in the base use case to insert the extension by placing a reference in the extending use case to a named extension point in the base use case. The extension use case can often be abstract, but it does not necessarily have to be.

Extend Relationship: RU e-st Example



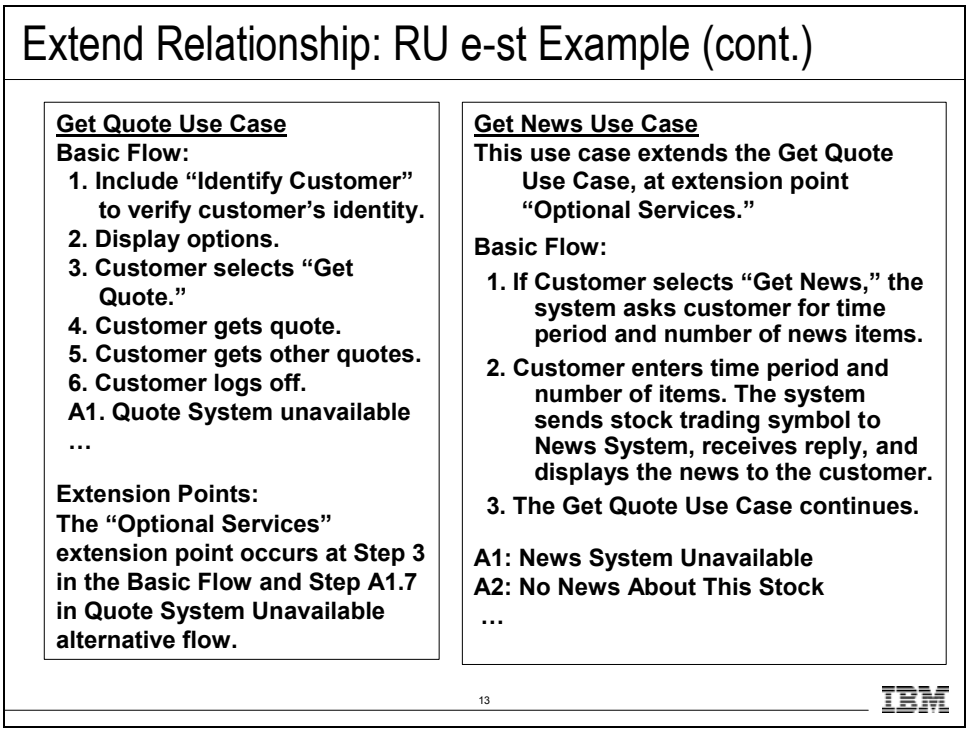
In this example, the Get Quote use case has been extended with other services, such as Get News and Get Expert Predictions. Both getting news and getting predictions are optional behaviors.

There are two reasons you would typically extend the behavior in this manner.

1. You are developing two versions of the system. The requirements specific to customer A are in one extending use case (Get News) and the requirements specific to customer B are in the other extending use case (Get Expert Predictions). The requirements common to both customers are in the extended use case (Get Quote).
2. You are developing generation two of a product, and you want to visually highlight where the changes in requirements are as well as make the updated requirements easier to identify.

Note: Overuse of «extend» and «include» leads to a model that is functionally decomposed and, therefore, harder to test and implement.

Extend Relationship: RU e-st Example (cont.)



In this example, the Get News use case extends the Get Quote use case.

The extension point "Optional Services" in the base use case (Get Quote) tells where the extending use case is performed. The extension point is a "hook" at which extending use cases can be attached. The base use case does not need to be modified with knowledge of the use cases that extend it. The "hook" is placed at a logical spot for extensions, whether or not there actually are any extensions.

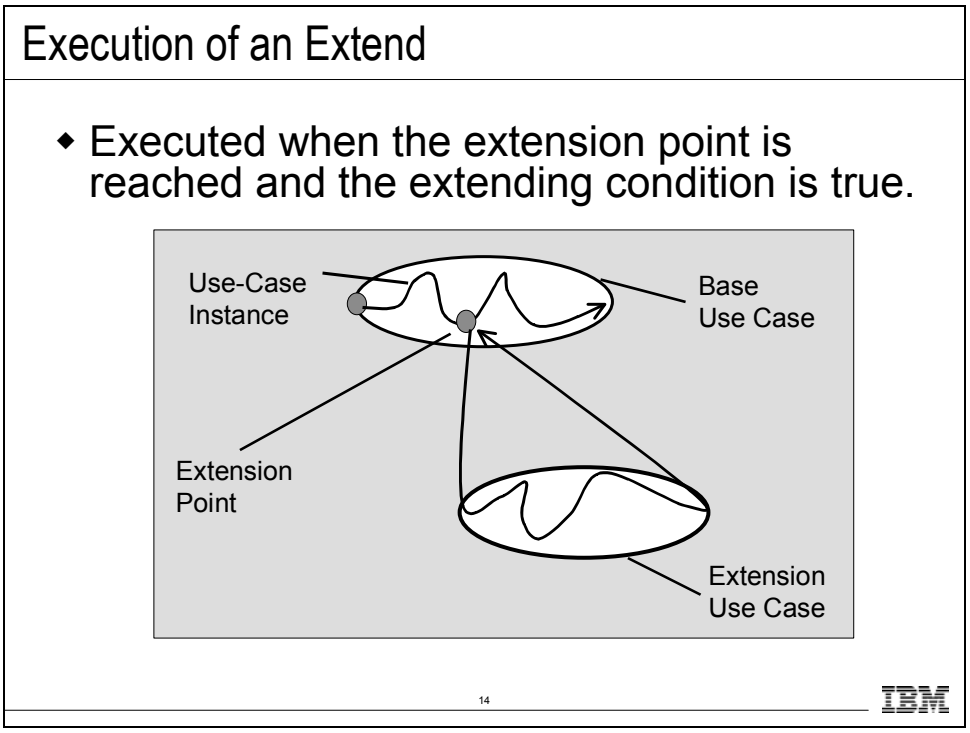
Using named extension points is a way of planning for the "dynamics of requirements." They are placed where you expect the requirements to change. The base use case does not need to know it is being extended.

The Get News use case references the extension point in the base use case. The extending use case needs to know and reference the extension point and other details in the base use case.

The Get News use case is only performed if the condition "If Customer selects 'Get News'" is true.

More than one use case can be attached at an extension point. In the example, it makes sense to attach both the Get News use case and the Get Expert Predictions use case at the extension point "Optional Services" in the base use case (Get Quote).

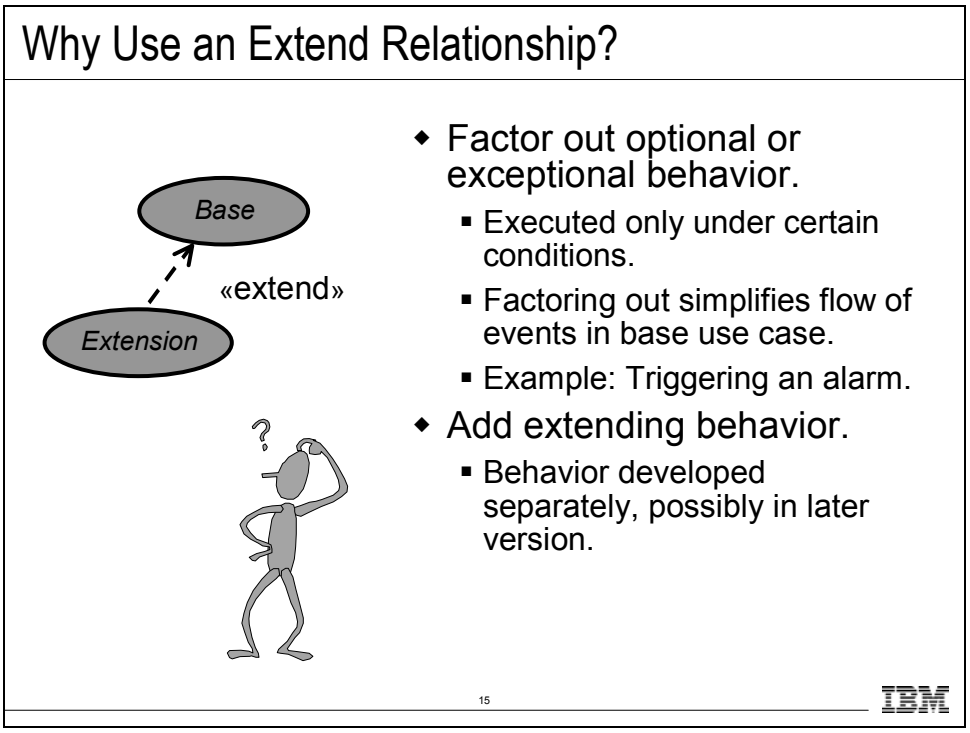
Execution of an Extend



When a use-case instance performing the base use case reaches a location in which the base use case has an extension point defined for it, the condition on the corresponding extend relationship is evaluated. If the condition is true or it is absent, the use-case instance follows the extension (or the insertion segment within it that corresponds to the extension point). If the condition of the extend relationship is false, the extension is not executed.

The extension use case may, just like any use case, have a basic flow of events and alternative flows of events. The exact path the use-case instance takes through the extension depends on what has happened before in the execution (the state of the use-case instance) and what happened in interaction with the actors as the extension was executed. Once the use-case instance has performed the extension, the use-case instance resumes executing the base use case at the point where it left off.

Why Use an Extend Relationship?



If there is a part of a base use case that is optional you can factor that part out to an addition use case in order to simplify the structure of the base use case. The addition is implicitly inserted in the base use case, using the «extend» relationship.

You can also use an «extend» relationship to show the changes and additions to requirements for different versions of the product. Version 2's requirements could be located in an extension use case, thereby making the requirement changes visually explicit.

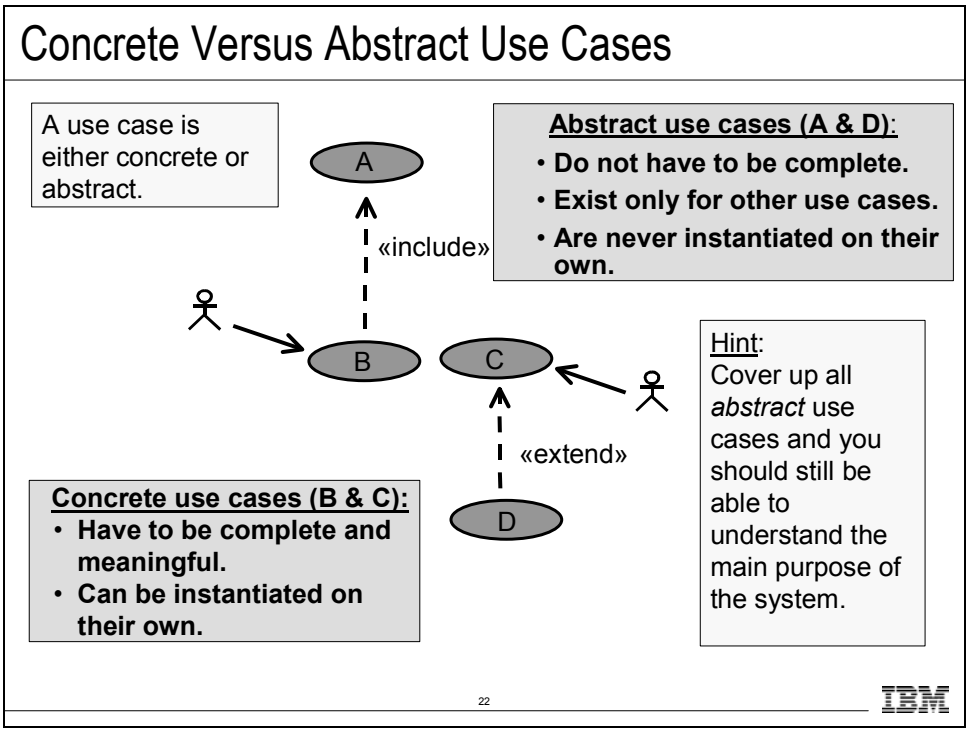
The extension is conditional, which means its execution is dependent on what has happened while executing the base use case. The base use case does not control the conditions for the execution of the extension. Those conditions are described within the extend relationship. The extension use case may access and modify properties of the base use case. The base use case, however, cannot see the extensions and may not access their properties.

The base use case is implicitly modified by the extensions. You can also say that the base use case defines a framework into which extensions can be added, but the base does not have any visibility of the specific extensions.

The base use case should be complete in and of itself, meaning that it should be understandable and meaningful without any references to the extensions. However, the base use case is not independent of the extensions because it cannot be executed without the possibility of following the extensions.

The base use case has no knowledge of the use case that extends it and, therefore, cannot see the properties of the extending use case. The extending use case knows which use case it extends and can see all the properties of the base use case.

Concrete Versus Abstract Use Cases



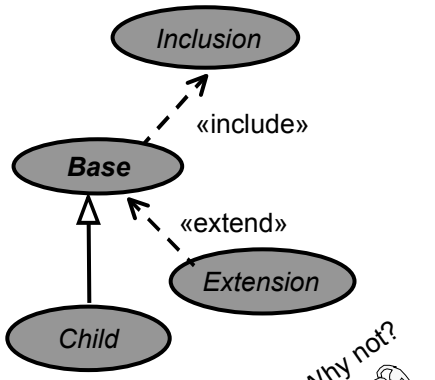
When you extract behavior into an included or extended use case, these new use cases are most often the kind that are never executed alone; they always exist as part of another use case. They are abstract use cases, but a concrete use case can:

- Include another concrete use case.
- Be extended by a concrete use case.

You may want to avoid this confusion by having guidelines that state that all extending and included use cases are abstract. Business decisions must be documented in your QA Plan.

Why Wouldn't You Structure The Model?

Why Wouldn't You Structure The Model?

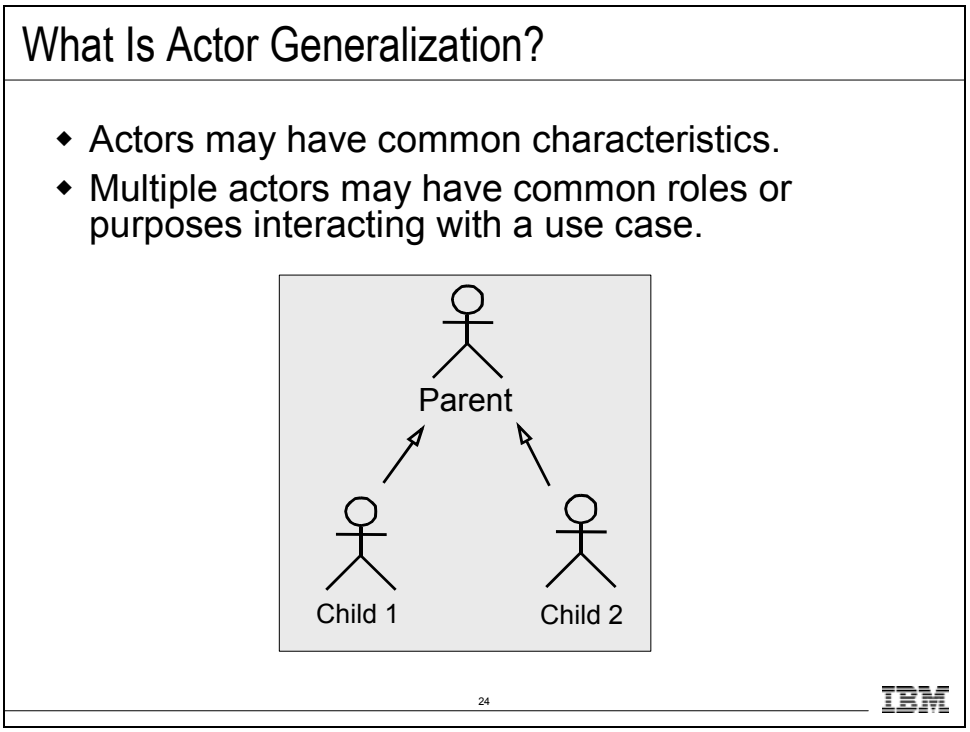


- The solution is harder to see when the use case gets fragmented.
  - Functionally decompose the requirements.
  - Decrease understandability.
  - Increase complexity.
  - Increases effort for reviewers, implementers and testers.
  - Not all stakeholders are comfortable with the format.
- The use-case model looks like a design.

23

IBM

What Is Actor Generalization?

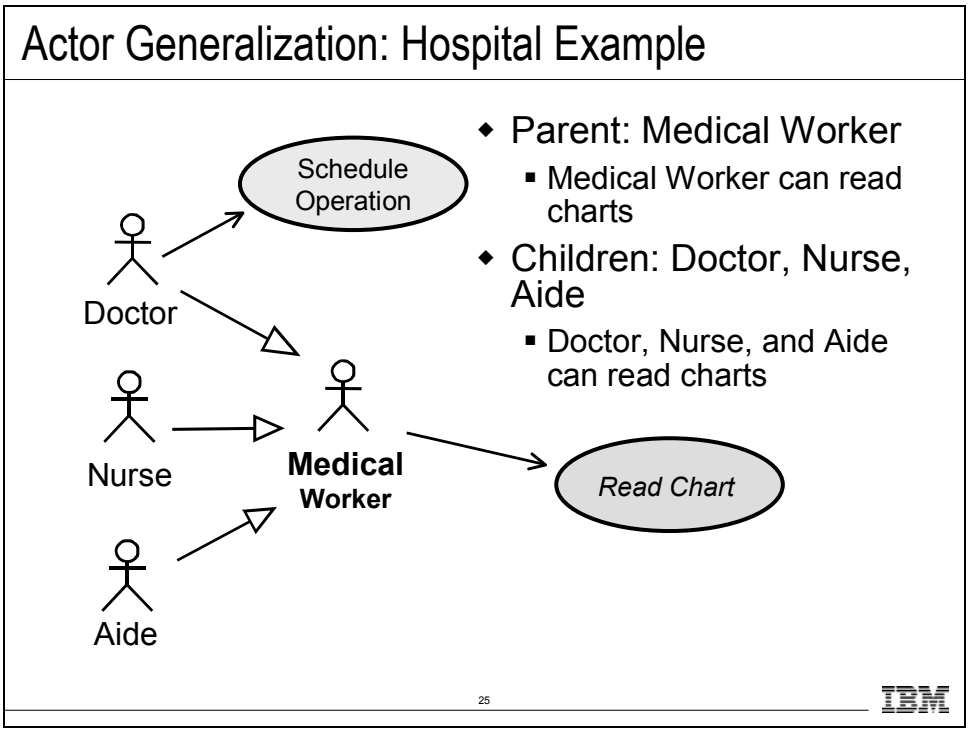


A user can play several roles in relation to the system, which means that the user may, in fact, correspond to several actors. To make the model clearer, you can represent relationships among actors. The shared role is modeled as a parent actor. The children actors inherit the common roles of the parent.

A parent actor may be specialized into one or more child actors that represent more specific forms of the parent. Neither parent nor child is necessarily abstract, although the parent in most cases is abstract. A child inherits all of the relationships of the parent. Children of the same parent are all specializations of the parent. This is generalization as applicable to actors.

A specific example on the following page.

Actor Generalization: Hospital Example



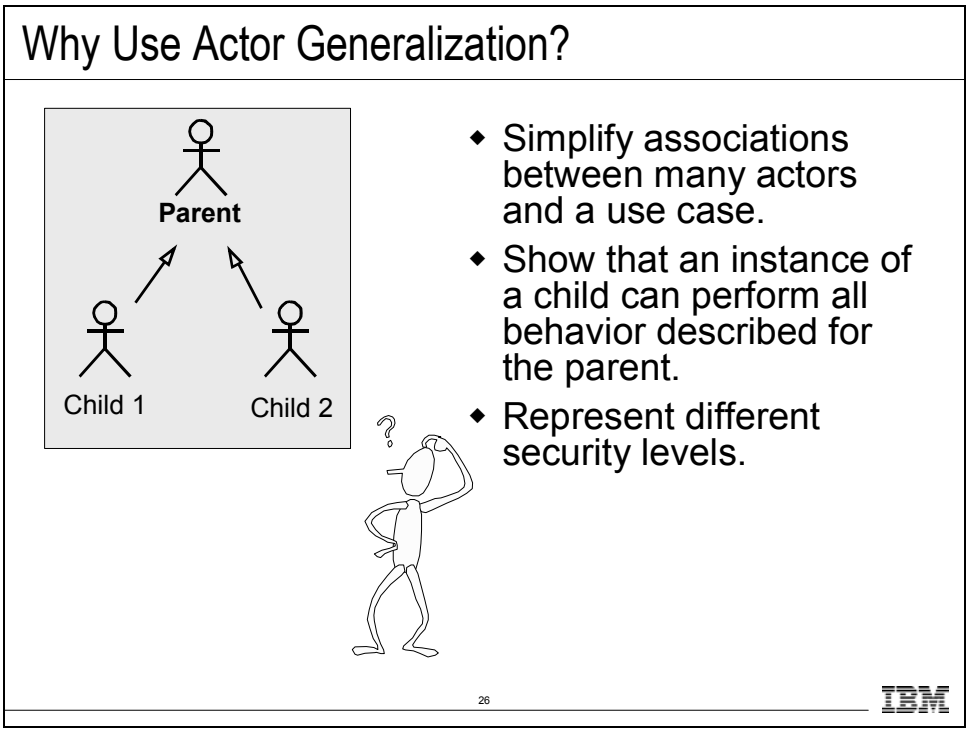
There are many actors (roles) that interact with a hospital record-keeping system, including doctors, nurses, and aides. Some use cases can be done by many actors. For example, there may be ten different actors in a hospital who can all read patient charts. You could have ten communicates-associations (represented by 10 lines) between each of the ten actors and the Read Chart use case. Or, you could simplify by defining all ten actors as children of a parent actor called Medical Worker.

Then, you only draw a single communicates-association between the Medical Worker and the Read Chart use case. All of the child actors, such as doctors, nurses, and aides, inherit the capability to do the Read Chart use case.

The medical worker actor participate in lots of other use cases, such as the Ring for Security use case and Record Temperature on Chart use case. The child actors, such as doctors, nurses, and aides, inherit the capability to do all of these use cases.

Why even have the child actors? Each actor (role) can participate in some other use cases that the other actors cannot. For example, only Doctor actors can perform the Schedule Operation use case. So, you draw a communicates-association between the Doctor actor and the Schedule Operation use case. You cannot draw a communicates-association between the Medical Worker actor and the Schedule Operation use case because the other children of the medical worker (such as nurses and aides) cannot perform the Schedule Operation use case.

Why Use Actor Generalization?

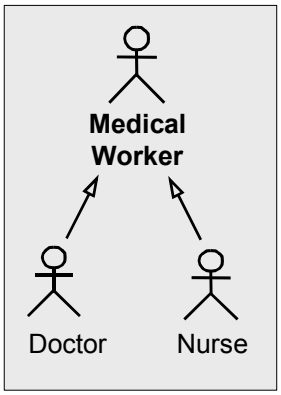


There are many uses for actor generalization. Specifically, there are two common reasons:

- Simplify the diagrams when multiple actors all perform the same use case. When you get many actors performing the same use case you end up with a "chop stock" effect of lines all pointing to the same use case. Usually you can identify some common role that each actor is performing, create an abstract actor for the role and then use a single communicates association.
- To show different security levels for actors participating in a use case. When used in conjunction with preconditions at the start of different flows, you can represent that certain flows can only be performed by certain actors.

Abstract Versus Concrete Actors

Abstract Versus Concrete Actors

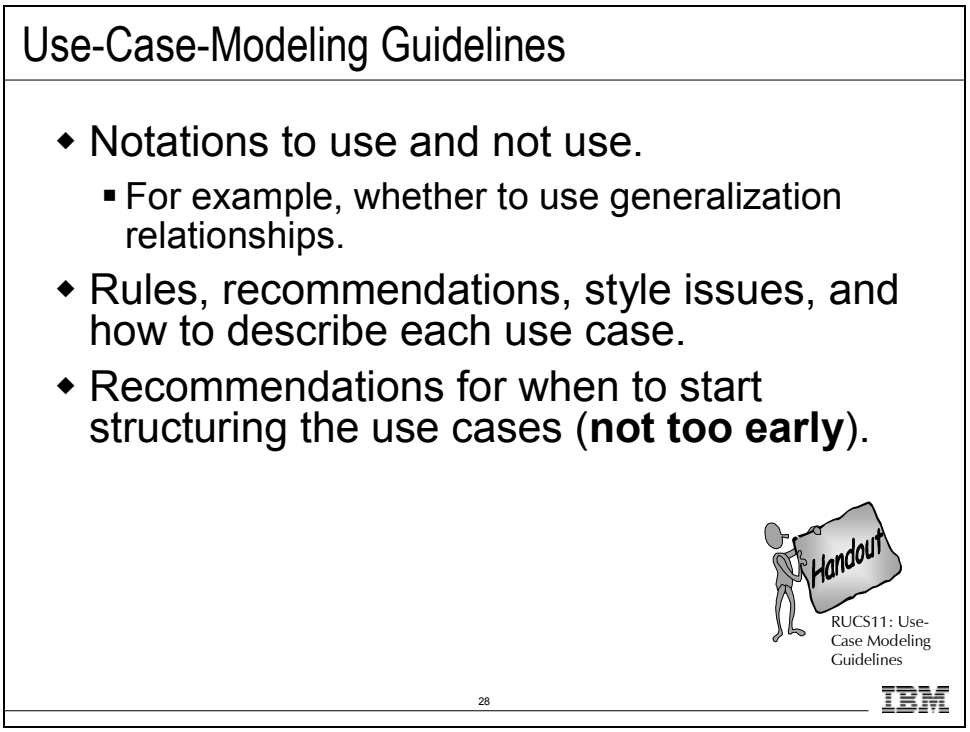


- An abstract actor contains the common part of the roles.
  - It cannot be instantiated itself.
  - Example:
    - No person is hired to be a Medical Worker.
- A concrete actor can be instantiated.
  - Example:
    - Lauren is a Doctor.
    - Daniel is a nurse.

27

IBM

Use-Case-Modeling Guidelines



Here is a proposed outline of a Use-Case-Modeling Guidelines document:

1. **Brief Description:**

A brief description of the role and purpose of the Use-Case-Modeling Guidelines.

2. **References:**

A description of related or referenced documents.

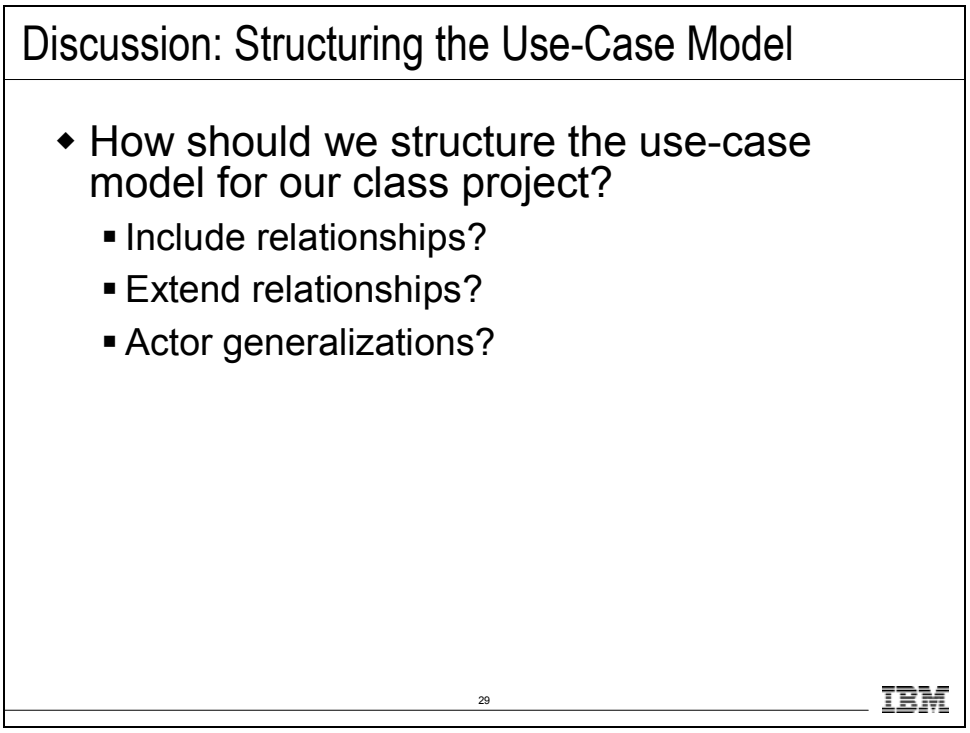
3. **General Use-Case-Modeling Guidelines:**

This section describes which notation to use in the use-case model. For example, you may have decided not to use extends-relationships between use cases.

4. **How to Describe a Use Case:**

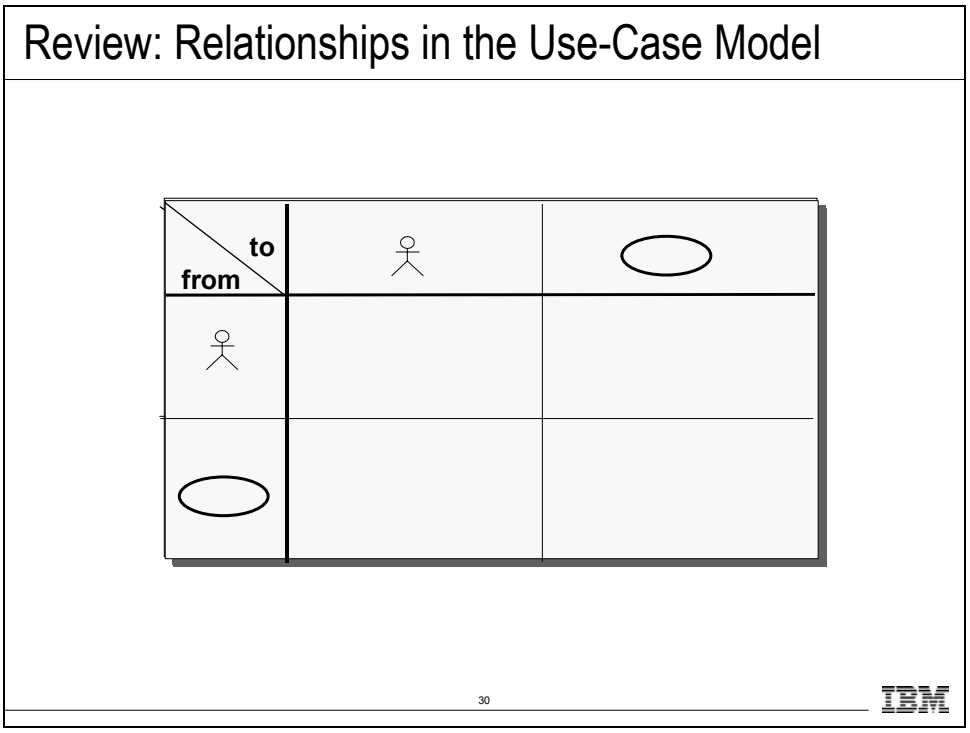
This section gives rules, recommendations, style issues, and how you should describe each use case.

Discussion: Structuring the Use-Case Model

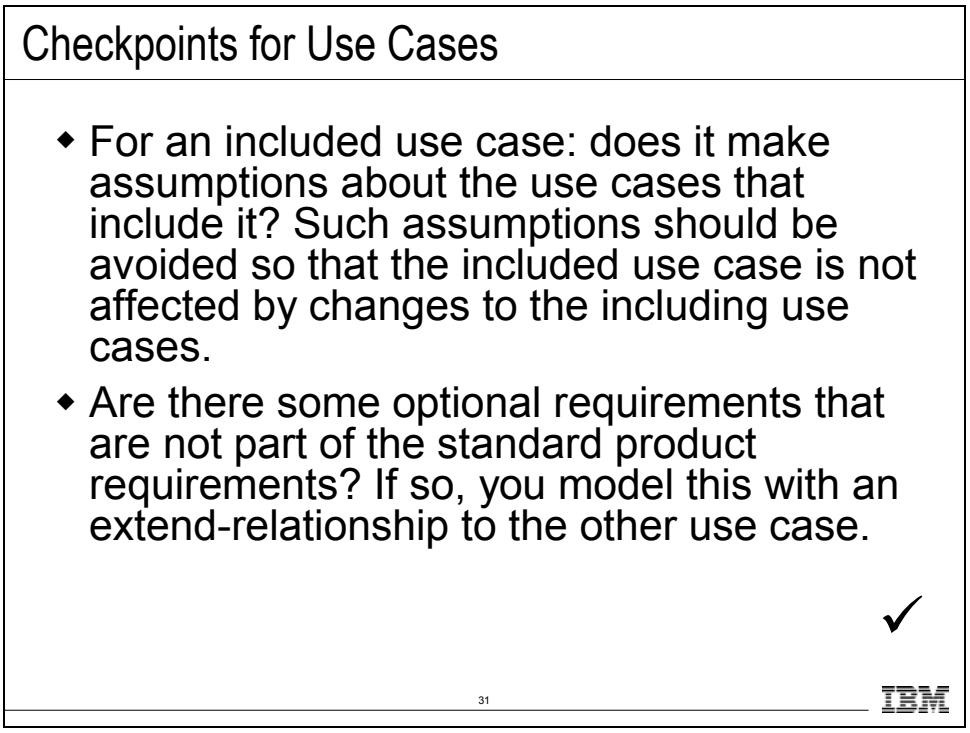


One possible candidate in the class project is to create an Identify Customer use case and include it in all use cases that require the user to log in to the system.

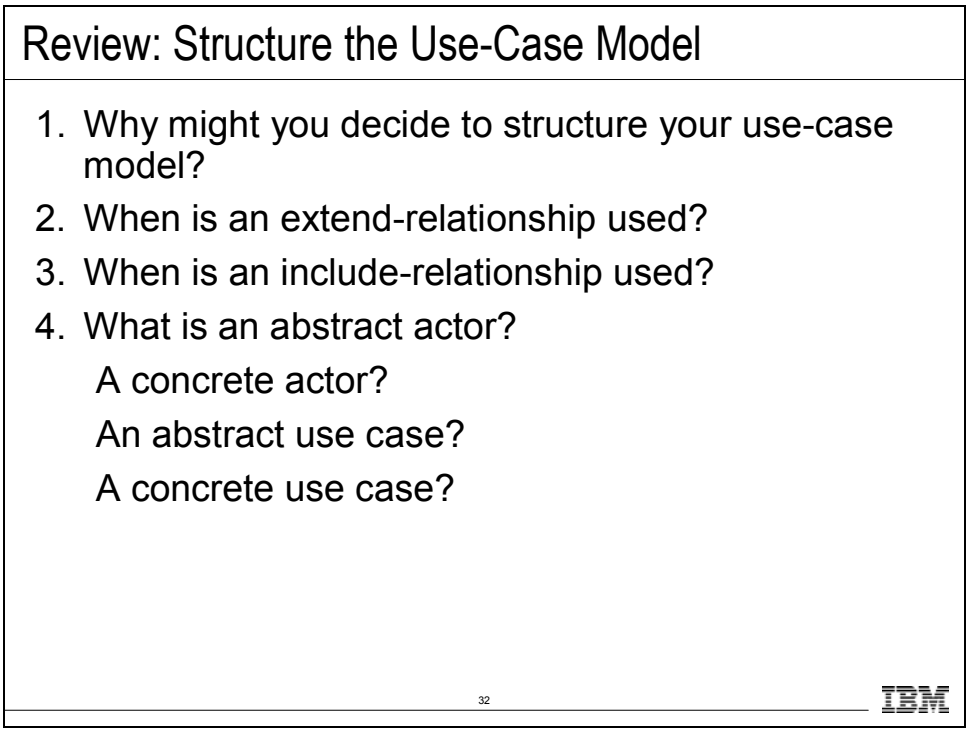
Review: Relationships in the Use-Case Model



Checkpoints for Use Cases



Review: Structure the Use-Case Model



1.

2.

3.

4.

Summary

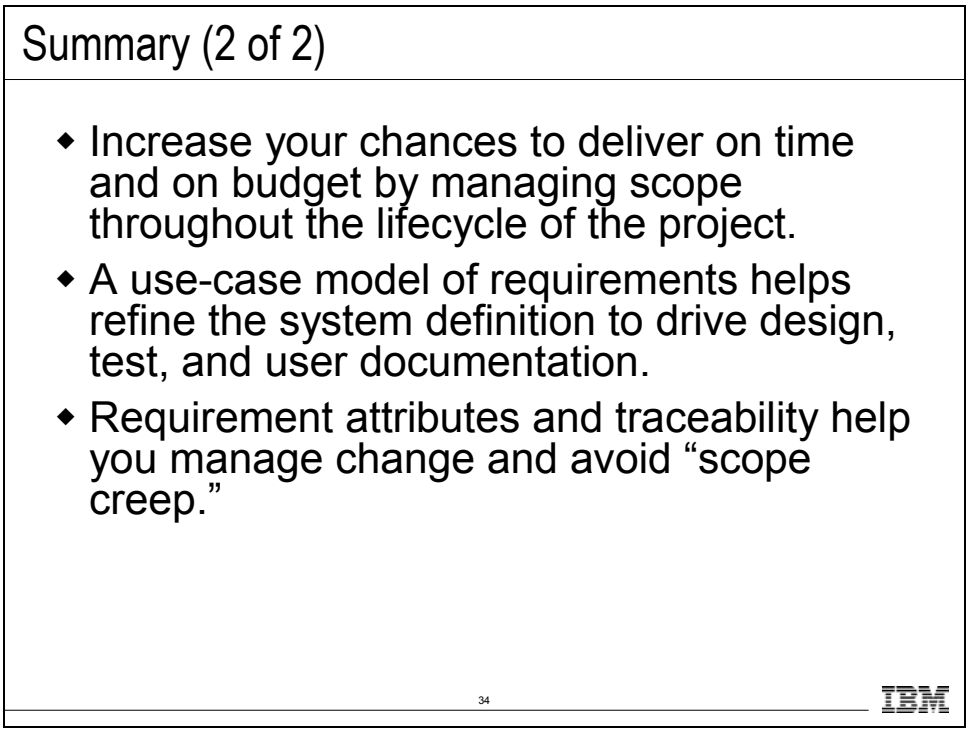
Summary (1 of 2)

- Build the right system right by using a process to define and manage requirements to meet the customer's needs.
- Effective problem analysis helps avoid the "Yes, but…"
- Elicitation helps you understand your stakeholders' needs.
- Use features and a use-case model to gain agreement with the customer on the definition of the system.

33

IBM

Summary



The goal in software development is to deliver quality products on time and on budget that meet the customer's real needs.

What did you learn that will help meet the goal?

MRMUC Handouts

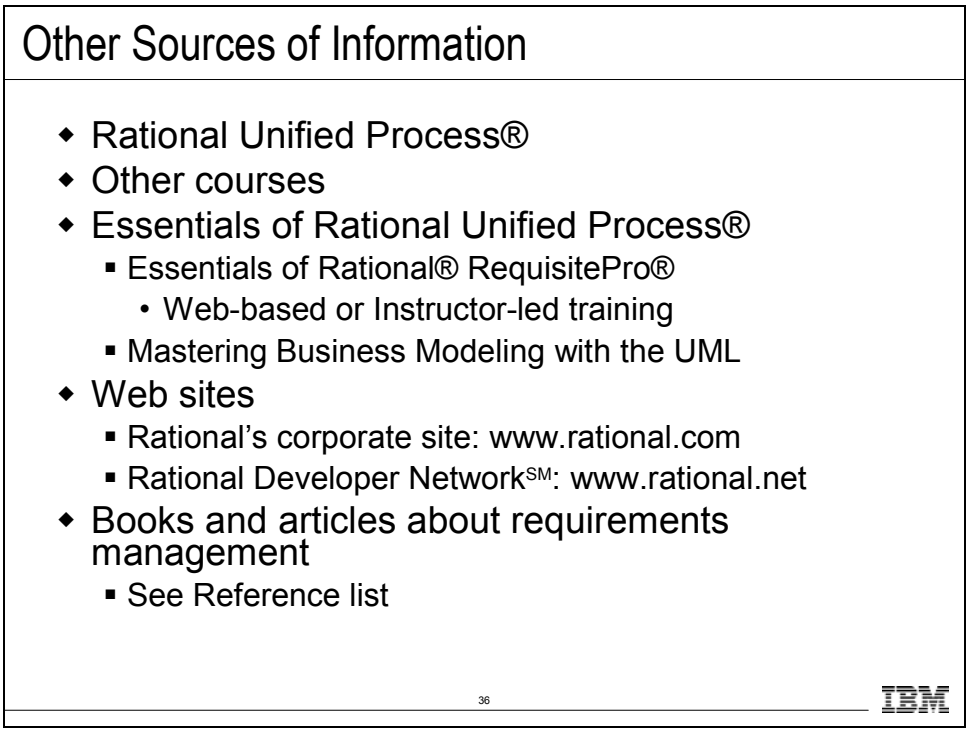
MRMUC Handouts

- WP 1: The Five Levels of Requirements Management Maturity
- WP 2: Features, Use Cases, and Requirements, Oh My!
- WP 3: Using the RUP to Evolve a Legacy System
- WP 4: Generating Test Cases from Use Cases
- WP 5: Structuring the Use-Case Model
- WP 6: ACRE: Selecting Methods For Requirements Acquisition

35

IBM

Other Sources of Information



There are many other sources of information about requirements management, including the sources listed here. Students who want more information about related topics should consider attending courses about the Rational Unified Process, Business Modeling, or RequisitePro.

Students who want to learn more about how to realize a use-case model and requirements in an object-oriented design should consider taking the Essentials of Visual Modeling with the UML or the Mastering Object-Oriented Analysis and Design courses.

Students who want to learn more about making reports that draw from the RequisitePro and Rational Rose repositories should consider taking a course in Rational SoDA® (Software Documentation Automation).

Students who want to learn more about quality testing from the beginning should consider taking Principles of Software Testing for Testers. Rational also offers courses on our automated testing tools that cover functional, performance, and embedded software testing.

Kurt Bittner and Ian Spence have written an excellent book that truly captures the essence of developing use cases. This book is a "must have" for anyone developing use cases. The book details are: *Use Case Modeling*, Kurt Bittner and Ian Spence, Addison Wesley, 2002, ISBN: 0-201-70913-9.

Dean Leffingwell and Don Widrig have written an excellent book on requirements management. The book details are: *Managing Software Requirements a Unified Approach*, D. Leffingwell and D. Widrig, Addison Wesley, 1999, ISBN: 0-201-61593-2.

Visit www.omg.org to learn more information about the UML.